19

BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

StampedLock: İyimser Okuma (Optimistic Reading) & Stamp Doğrulama

StampedLock: Optimistic Reading, Stamp Validation & Upgrades

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
                STAMPEDLOCK: İYİMSER OKUMA (OPTIMISTIC READ) DÖNGÜSÜ

 1. İYİMSER OKUMA BAŞLAT:
    long stamp = sl.tryOptimisticRead(); // SIFIR KİLİT! Bellek durumu snapshot'ı al.

 2. YEREL DEĞİŞKENLERE KOPYALA:
    double currentX = x;
    double currentY = y;

 3. STAMP'I DOĞRULA (validate):
    if (!sl.validate(stamp)) { ──► "Biri araya girip yazdı mı?"
        // EVET! Araya bir yazıcı girdi, veriler tutarsız olabilir!
        // PES ETME: Karamsar (Pessimistic) Okuma Kilidine geç:
        stamp = sl.readLock();
        try {
            currentX = x;
            currentY = y;
        } finally {
            sl.unlockRead(stamp);
        }
    }
 AVANTAJ: Yazıcı yoksa %100 KİLİTSİZ ve sıfır CAS maliyetiyle okuma tamamlanır!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir müzedeki tarihi bir saatin akrep ve yelkovanına bakmak istiyorsunuz. Normal sistemde saatçiyi durdurup saati kilitlersiniz. StampedLock'un 'İyimser Okuması'nda ise saati hiç kilitlemezsiniz! Yanından geçerken akrep ile yelkovana hızlıca göz ucuyla bakarsınız (tryOptimisticRead). Sonra elinizdeki bilete bakıp sorarsınız: 'Ben bakarken saatçi saati kurcaladı mı?' (validate). Saatçi dokunmadıysa saati öğrendiniz, bitti! Dokunduysa hemen geri döner, bu sefer saati durdurup garantiye alarak okursunuz!

🇬🇧 ENGLISH (ELI5)

Imagine checking the time on a grand clock. Under standard locking, you halt the clockwork mechanism while looking. StampedLock's 'Optimistic Read' never halts the clockwork! You glance at the hands as you walk past (`tryOptimisticRead`). Then you check your ticket validation stamp: 'Did the clockmaker wind the hands while I was glancing?' (`validate`). If not, your read is mathematically valid with ZERO lock overhead! If the stamp invalidated, you gracefully fall back to a full pessimistic lock.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

StampedLock Mimarisi ve Reentrancy Uyarısı: 1) İyimser Okuma (Optimistic Read): `tryOptimisticRead()` bir kilit almaz; yalnızca sürüm numarasını (Stamp) döner. Okuma sırasında herhangi bir yazıcı thread `writeLock()` alırsa stamp sürümü değişir ve `validate(stamp)` false döner. 2) ReentrantReadWriteLock'tan Farkı: RWLock'ta okuyucular yazıcıları tamamen bloke eder; bu da 'Writer Starvation' (yazıcıların aç kalması) sorununa yol açar. StampedLock'ta iyimser okuyucular yazıcıları ASLA bloke etmez! 3) ⚠️ EN BÜYÜK TUZAK (Reentrant DEĞİLDİR!): StampedLock `Reentrant` DEĞİLDİR! Bir thread tuttuğu `readLock()` varken tekrar `readLock()` almaya çalışırsa veya kilit içinde kendini çağıran metotlar varsa DEADLOCK meydana gelir!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

StampedLock Internal Topology: 1) Stamp Encoding: StampedLock encodes lock state as a 64-bit integer combining sequence bits and reader counts. An optimistic read returns the current sequence stamp with zero memory writes. 2) Writer Starvation Immunity: In `ReentrantReadWriteLock`, continuous readers starve pending writers indefinitely. StampedLock allows writers to acquire the exclusive lock instantly, invalidating optimistic readers without stalling. 3) ⚠️ Critical Fatal Constraint: StampedLock is NON-REENTRANT! If a thread holding a read or write lock calls another method attempting to acquire the same StampedLock, it deadlocks itself immediately. It also does NOT implement the standard `Lock` interface.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: StampedLock ile İyimser Okumalı 2D Nokta Modeli
import java.util.concurrent.locks.StampedLock;

public class Point2D {
    private double x, y;
    private final StampedLock sl = new StampedLock();

    public void move(double deltaX, double deltaY) {
        long stamp = sl.writeLock(); // Karamsar Yazma Kilidi
        try {
            x += deltaX;
            y += deltaY;
        } finally {
            sl.unlockWrite(stamp);
        }
    }

    public double distanceFromOrigin() {
        // 1. İyimser Okuma Dene (Kilit YOK!)
        long stamp = sl.tryOptimisticRead();
        double curX = x, curY = y;

        // 2. Araya yazıcı girdi mi kontrol et
        if (!sl.validate(stamp)) {
            // Yazıcı girmiş! Karamsar okuma kilidine düş:
            stamp = sl.readLock();
            try {
                curX = x;
                curY = y;
            } finally {
                sl.unlockRead(stamp);
            }
        }
        return Math.sqrt(curX * curX + curY * curY);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı StampedLock Sorusu: 'StampedLock reentrant mıdır?' Yanıt: 'HAYIR! Kesinlikle reentrant değildir. Aynı thread aynı kilide ikinci kez girmeye çalışırsa anında self-deadlock yaşanır!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mandatory StampedLock Interview Check: 'Is StampedLock reentrant?' Senior Answer: 'NO! StampedLock is strictly NON-REENTRANT. Attempting reentrant acquisition on the same thread will permanently deadlock the application.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- StampedLock iyimser okumada hiçbir kilit almaz; yazıcıları asla bloke etmez.
- validate(stamp) başarısız olursa karamsar readLock'a geri dönülür.
- StampedLock REENTRANT DEĞİLDİR; kendini çağıran kilitler deadlock olur.

🎯 Key Takeaways for the Interview

- Optimistic reads in StampedLock acquire no lock and never block writers.
- If validate() fails, execution falls back gracefully to a pessimistic readLock.
- StampedLock is NOT reentrant; self-acquisitions cause instant deadlocks.